



Arrays

By: Mir Faraz Ali
Programming Fundamentals



What is an Array?

Definition:

An array is a collection of multiple values of the same data type, stored in continuous memory locations.

Why do we use arrays?

- To store many values in one variable
- Easier searching & updating
- Efficient use of memory
- Can be accessed quickly using index numbers

Accessing Array Elements Using Index:

- We use an index number to access any element in an array.

Syntax:

- `arrayName[index];`

Example:

```
int marks[5] = {80, 75, 90, 66, 88};  
cout << marks[2]; // Output: 90
```

Using Loops to Access Arrays:

Loops help us access array elements one by one.

Example:

```
int marks[5] = {80, 75, 90, 66, 88};
```

```
for (int i = 0; i < 5; i++) {  
    cout << "Marks: " << marks[i] << endl;  
}
```

Example:

Why use loops?

- Saves time
- Less code
- Works on any size of array

Visualization:

$i = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Loop moves through each index.

Arrays in Functions:

- Arrays are often used with functions.
- We can use arrays in two ways inside functions:
 1. Passing individual array elements to a function
 - (using an index $\rightarrow$ `arr[2]`)
 2. Passing the whole array to a function
 - (using array name $\rightarrow$ `arr`)

Why do we use arrays in functions?

- To process multiple values
- To avoid repeating code
- To compute totals, averages, maximum value, etc.
- To make programs structured and organized

Indexed Variables as Function Arguments:

This means sending one element of the array to a function.

Example:

```
void display(int value) {  
    cout << "Value received = " << value;  
}  
  
int main() {  
    int numbers[3] = {10, 20, 30};  
    display(numbers[1]); // Sending element at index 1 (20)  
}
```


Functions That Return an Array:

In C++, a function cannot directly return an entire array, but we can still return array-like data using:

- ✓ A pointer
- ✓ OR by filling an array passed to the function

Filling the array inside the function:

```
void fillArray(int arr[], int size) {  
    for(int i = 0; i < size; i++) {  
        arr[i] = i + 1;    // Fill array with 1,2,3...  
    }  
}  
  
int main() {  
    int data[5];  
    fillArray(data, 5);  
}
```

Partially Filled Arrays:

A partially filled array is an array where only some positions have valid data.

```
int data[10];  
int used = 4; // Only 4 entries are used
```

```
data[0] = 5;  
data[1] = 12;  
data[2] = 7;  
data[3] = 9;
```

Why do we use partially filled arrays?

- When we don't know how many items will be entered
- Useful for storing list of names, marks, IDs
- Helps avoid errors when accessing empty elements

Real-life example:

Imagine a parking lot with 50 spaces (array size = 50).

But only 23 cars are parked (used = 23).

The remaining spaces are empty (unused indexes).

Searching in Arrays:

- What is Searching?
- Searching means looking for a specific value inside an array and finding its index or confirming whether it exists.
- Example:
- Finding “30” inside an array of numbers.

Types of Searching:

- Linear Search (Sequential Search):
 - Linear Search is a method in which we check each element one by one from start to end until we find the required value.

How Linear Search Works:

- Start from index 0
- Compare each element with the value you want to find
- If found → return the index
- If not found → continue until the last element

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
int search = 30;
bool found = false;
for(int i = 0; i < 5; i++) {
    if(arr[i] == search) {
        cout << "Value found at index " << i;
        found = true;
        break;
    }
}
if(!found) {
    cout << "Value not found";
}
```


When to Use Linear Search?

- Array is small
- Array is not sorted
- Easy to implement
- Works with numbers, characters, or strings



End Of Class

